



Universidad Andrés Bello

Magíster en Ciencia de Datos e Inteligencia Artificial

MCDI500 — Programación para la Ciencia de Datos

Avance Fase 3 — Semana 2

Preparación Reproducible de Órdenes de Compra Ágil de Valparaíso

Grupo 7:

Benjamín Araya Matta

Alan Cañete Calquin

Patricio Cortez Triviño

Jorge Guaico Ortega

Docente: Omar Salinas

Fecha de entrega: 25 de septiembre de 2026

Repositorio: <https://github.com/BenjaArayaM/proyecto-grupo7-mcdi500>

Contenido

Codificación funcional y arquitectura básica.....	3
Preprocesamiento y transformación.....	4
Validación técnica y verificación.....	5
Eficiencia y optimización	7
Diseño estructurado del código.....	10
Implementación modular y organización del código.....	10
Documentación de arquitectura	11
Notebooks ejecutables	12
Repositorio GitHub.....	13
Referencias.....	14

Codificación funcional y arquitectura básica

El avance de la Fase 3 fue desarrollado por el conjunto de datos procesados en la etapa anterior, buscando mantener la continuidad y reproducibilidad del flujo de trabajo. Para esta fase se creó un notebook específico el cual está dirigido al análisis de la eficiencia algorítmica mediante el método del rango intercuartílico (IQR), utilizando como variable de análisis MontoNetoOC_CLP.

La estructura inicial del código separa las principales etapas del proceso, que son la identificación de la raíz del repositorio, la carga del conjunto de datos procesados, la selección y validación de las variables de interés y el cálculo de los estadísticos necesarios para determinar los límites del método IQR. Esta separación de etapas tiene por finalidad mantener una correcta comprensión desde la entrada de los datos hasta que se obtienen los resultados, facilitando de esta manera la incorporación de funciones independientes y reutilizables.

Como parte de la organización de la Fase 3, se incorpora al repositorio las carpetas F3/notebooks, F3/src, F3/tests y F3/docs. El objetivo de esta estructura es la de facilitar la modularización del código y su posterior evolución en las etapas posteriores. La tabla 1 resume la función asignada a cada componente.

Tabla 1

Estructura inicial fase 3

Componente	Función dentro de la fase 3
F3/notebooks	Contiene el notebook utilizado para ejecutar y documentar el análisis

F3/src	Destinado a las funciones reutilizables y a la modularización del código
F3/tests	Destinado a las pruebas y validación de las funciones implementadas
F3/docs	Contiene documentación complementaria y antecedentes utilizados en el informe

Nota. Elaboración propia a partir de la estructura del repositorio del proyecto

Preprocesamiento y transformación

Para el desarrollo de la Fase 3 utilizamos como punto de partida el archivo ya procesado en la semana anterior *F2/data/processed/ordenes.csv*. De esta forma, no se repitieron las labores de limpieza y transformación ya realizadas, manteniendo la continuidad del pipeline, evitando modificar de manera innecesaria los datos que ya habían sido validados.

Para el análisis de eficiencia algorítmica seleccionamos la variable *MontoNetoOC_CLP*, el cual corresponde al monto neto de cada orden de compra expresado en pesos chilenos. Antes de aplicar el método IQR, la variable se convirtió a formato numérico con control de errores y se verificó la cantidad de observaciones disponibles, valores válidos y datos faltantes.

La validación arrojó 541 observaciones, dentro de las cuales 541 corresponden a valores numéricos válidos y ninguna presenta valores faltantes. Por este motivo, no se realizaron métodos de imputación ni se eliminaron registros para efectuar el análisis. La conservación de las observaciones permite que las implementaciones algorítmicas siguientes actúen sobre el mismo conjunto de datos y bajo condiciones comparables.

Con la variable seleccionada se calcularon el primer cuartil (Q1), el tercer cuartil (Q3) y el rango intercuartílico (IQR). Estos valores estadísticos nos permiten establecer los límites superior e inferior, que se utilizan para identificar posibles valores atípicos. En esta etapa, estos valores no son eliminados del conjunto de datos, sino que se identifican para su análisis posterior y para comparar las distintas implementaciones del algoritmo.

Tabla 2

Validación de la variable utilizada en fase 3

Indicador	Resultado
Variable analizada	<i>MontoNetoOC_CLP</i>
Observaciones	541
Valores numéricos válidos	541
Valores faltantes	0
Registros eliminados	0
Registros imputados	0

Nota. Elaboración propia a partir de la validación del archivo procesado generado en la

Fase 2

Validación técnica y verificación

La validación técnica se realizó por medio de la ejecución secuencial del notebook desarrollado para la Fase 3, verificando inicialmente que el conjunto de datos y la variable *MontoNetoOC_CLP* estén correctamente cargados. Luego, se comprobaron los resultados obtenidos durante el cálculo del rango intercuartílico y la identificación de valores potencialmente atípicos.

La ejecución permitió obtener el primer cuartil (Q1) por un valor de \$217.923, el tercer cuartil (Q3) por un valor de \$1.620.152 y un rango intercuartílico (IQR) de \$1.402.229. Con estos valores se estableció un límite superior de \$3.723.495,50, al utilizar como criterio de 1,5 veces IQR. Como resultados, se obtuvieron 61 observaciones sobre dicho límite.

Los resultados obtenidos son consistentes con los antecedentes que se generaron durante la Fase 2, donde también fueron identificados 61 órdenes por sobre el límite superior de \$3.723.495,50. Esta semejanza nos permite verificar que el procedimiento llevado a cabo en la fase 3 reproduce el criterio previamente utilizado y mantiene la continuidad entre ambas etapas del proyecto.

Las observaciones detectadas como potencialmente atípicas se mantienen dentro del conjunto de datos. Su clasificación por medio de IQR corresponde a un criterio estadístico y no implica que los registros sean erróneos. Esta decisión nos permite conservar la información original y utilizar el mismo conjunto de observaciones en las siguientes comparaciones de eficiencia algorítmica.

Tabla 3

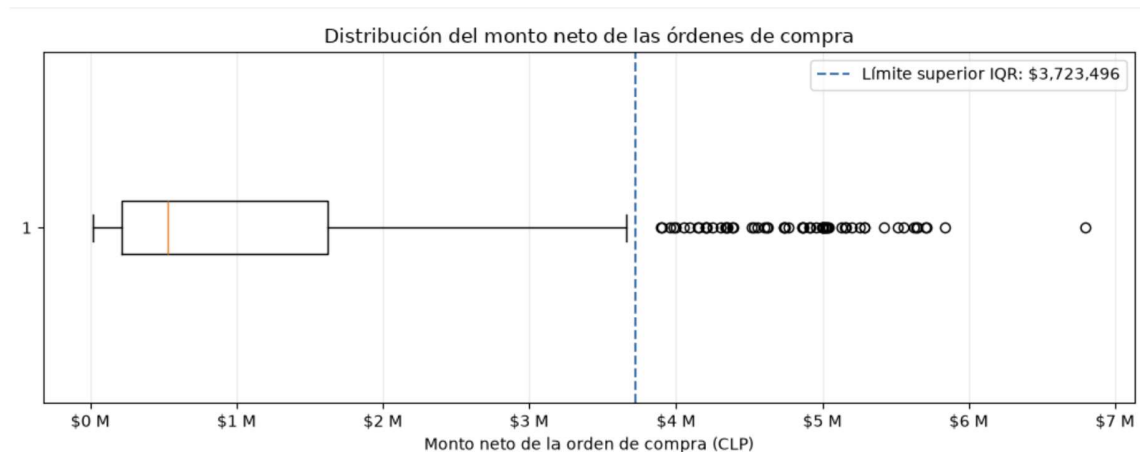
Resultados de la validación mediante el método IQR

Indicador	Resultado
Observaciones analizadas	541
Primer cuartil (Q1)	\$217.923
Tercer cuartil (Q3)	\$1.620.152
Rango intercuartílico (IQR)	\$1.402.229
Límite superior	\$3.723.495,50
Observaciones sobre el límite superior	61
Observaciones eliminadas	0

Nota. Elaboración propia a partir de los resultados obtenidos mediante ejecución del notebook de la Fase 3

Figura 1

Distribución del monto neto de las órdenes de compra mediante el método IQR



Nota. La línea discontinua representa el límite superior calculado mediante criterio de 1,5 veces el rango intercuartílico (\$3.723.495,50). Los puntos situados sobre este límite corresponden a observaciones potencialmente atípicas, las que se mantienen en el conjunto de datos.

Eficiencia y optimización

Para evaluar la eficiencia del procedimiento de detección de valores atípicos mediante el método IQR se compararon dos implementaciones funcionales equivalentes, la primera utiliza un bucle para recorrer los valores de la variable *MontoNetoOC_CLP*, mientras que la segunda aplica una operación vectorizada mediante pandas.

Antes de comparar los tiempos de ejecución, se verificó que ambas implementaciones entregaran el mismo resultado. En ambos casos se identificaron 61 observaciones potencialmente atípicas sobre un total de 541 registros analizados. Esta comprobación permitió asegurar que la comparación se realiza bajo condiciones similares y las diferencias observadas corresponden al rendimiento de cada implementación y no a variaciones en los resultados.

Para realizar la medición se utilizó *time.perf_counter()*. Debido a que cada ejecución individual requiere de un tiempo muy breve, cada función fue ejecutada 1.000 veces dentro de cada medición. Se realizaron tres mediciones por implementación y se conservó el menor de los tiempos registrados, con el objetivo de reducir el efecto de procesos externos del computador.

En la ejecución final del notebook, la implementación por bucle obtuvo un mejor tiempo aproximado de 0,033 segundos para 1.000 ejecuciones, mientras que la versión vectorizada alcanzó aproximadamente 0,069 segundo. Aunque inicialmente se esperaba que la operación vectorizada presentara un mejor rendimiento, para el tamaño actual del conjunto de datos el bucle resultó más rápido. Este resultado demuestra la importancia de respaldar las decisiones de diseño mediante mediciones empíricas y no asumir que una estrategia será siempre más eficiente. En este prototipo se adopta la implementación mediante bucle como alternativa principal, ya que presentó el menor tiempo de ejecución manteniendo el mismo resultado que la versión vectorizada. Esta decisión puede ser reevaluada en etapas posteriores si aumenta el volumen de los datos.

Tabla 4

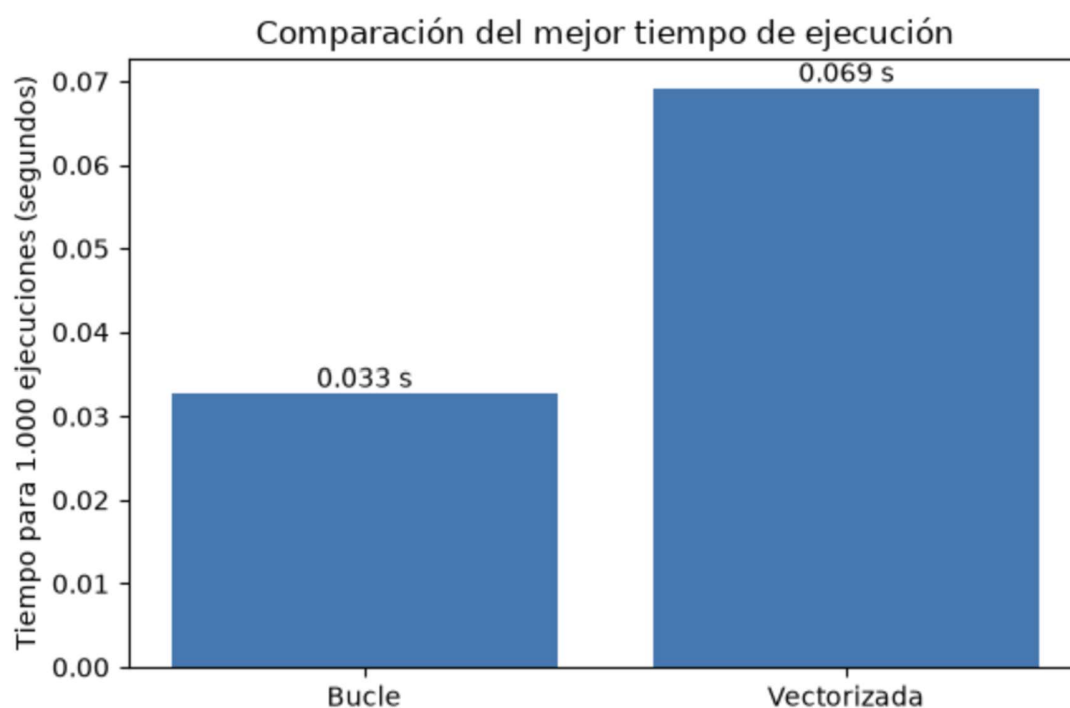
Comparación de tiempos de ejecución de las implementaciones

Implementación	Observaciones detectadas	Mejor tiempo para 1.000 ejecuciones
Bucle	61	0,033 s
Vectorizada	61	0,069 s

Nota.Elaboración propia a partir de las mediciones realizadas en el notebook de la Fase 3

Figura 2

Comparación del mejor tiempo de ejecución



Nota. La implementación mediante bucle presentó el menor tiempo de ejecución para el conjunto analizado de 541 observaciones. Ambas implementaciones entregaron el mismo resultado, identificando 61 valores potencialmente atípicos.

Diseño estructurado del código

Para la detección de valores atípicos mediante el método IQR se utilizó un enfoque de programación estructurada. El procedimiento se basa en evaluar una serie de valores respecto a límites previamente calculados, por lo que el problema tiene una secuencia de pasos definida y no es necesario recorrer estructuras anidadas o de profundidad desconocida. Por este motivo no se implementaron funciones recursivas ya que estas no aportan una ventaja clara, ya que el problema puede resolverse mediante una estructura iterativa simple y de fácil lectura. El uso de un bucle permite recorrer los valores de la variable *MontoNetoOC_CLP* y comprobar si cada observación se encuentra fuera de los límites establecidos por el método IQR.

La decisión de usar programación estructurada también facilita la trazabilidad del procedimiento, debido a que cada etapa puede seguirse de manera secuencial desde la carga de los datos, el cálculo de los cuartiles y límites, hasta la detección y validación de los valores atípicos. Además, la lógica de detección fue separada en funciones independientes, lo que permite probar cada implementación por separado y comparar sus resultados bajo las mismas condiciones. Esta organización mejora la legibilidad del código y facilita su reutilización en etapas posteriores.

La elección de este enfoque responde a la naturaleza del problema analizado y no a una limitación técnica. Si en fases posteriores se incorporan estructuras jerárquicas o procesos de profundidad variable, podría reevaluarse el uso de técnicas recursivas.

Implementación modular y organización del código

Durante el desarrollo de la Fase 3 se buscó reducir la concentración de lógica dentro del notebook y avanzar hacia una estructura de código más modular, para este objetivo las funciones utilizadas en la detección de valores atípicos mediante IQR fueron trasladadas al archivo *F3/src/funciones_iqr.py*, desde donde posteriormente se importan para ser utilizadas en el notebook principal. Esta separación contribuye a que el notebook se concentre en la carga de datos, ejecución, comparación e interpretación de resultados, mientras que la lógica reutilizable queda almacenada en un modulo independiente con la finalidad de evitar duplicar definiciones de funciones dentro del cuaderno y facilitando su reutilización en otras partes del proyecto.

El modulo posee dos funciones principales, una implementación basada en bucles y otra basada en operaciones vectorizadas con pandas, ambas reciben como entrada la serie de datos y los límites inferior y superior previamente calculados mediante IQR, devolviendo los valores que se encuentran fuera de dichos límites.

En esta etapa no se implementaron clases, ya que las funciones utilizadas no necesitan mantener estado entre las distintas ejecuciones. Para el problema trabajado, una estructura basada en funciones resulta suficiente y mantiene el código simple y comprensible, la guía de la evaluación formativa permite este enfoque, ya que el objetivo en esta etapa es avanzar en modularización y organización del código, mientras que la incorporación más completa de programación orientada a objetos se desarrollará posteriormente.

Documentación de arquitectura

La arquitectura utilizada en esta fase busca separar las distintas responsabilidades del proyecto y evitar que la lógica quede concentrada dentro de un único notebook, para esto la Fase 3 se

organiza por medio de carpetas y archivos con funciones específicas buscando mantener la continuidad del trabajo desarrollado en la Fase 2.

El notebook principal está en *F3/notebooks* y concentra la ejecución del análisis, la carga de datos procesados, la validación de resultados, las mediciones de eficiencia y su interpretación; por otra parte, la carpeta *F3/src* posee las funciones reutilizables asociadas al análisis algorítmico, específicamente el archivo *funciones_iqr.py* donde se implementan las versiones mediante bucle y operaciones vectorizadas para la detección de valores atípicos.

La separación entre notebook y modulo permite distinguir la lógica reutilizable de la presentación y análisis de resultados lo que reduce la duplicación del código y contribuye a que las funciones puedan ser reutilizadas, probadas o modificadas sin afectar la estructura general del cuaderno.

Como criterio de diseño, se privilegio una estructura simple y modular, adecuada al tamaño del proyecto. En fases posteriores, esta organización puede ampliarse mediante clases u otros componentes de programación orientada a objetos sin necesidad de modificar completamente la lógica ya implementada.

Notebooks ejecutables

El notebook *F3_Formativa_Eficiencia_IQR.ipynb* fue ejecutado de manera completa mediante la opción Restart Kernel and Run All Cells, verificando que las celdas se ejecutaran en orden y sin errores.

El cuaderno posee las salidas necesarias para respaldar los resultados obtenidos, incluyendo la validación de los datos, el cálculo IQR, la identificación de 61 valores potencialmente atípicos, la comparación de las dos implementaciones y las mediciones de tiempo de ejecución. Además, se incorporaron celdas Markdown para explicar las principales etapas del procedimiento y se incluyó una vinculación con las decisiones discutidas en el foro técnico de la Fase 3.

Repositorio GitHub

El desarrollo de la Fase 3 se mantiene en el repositorio del Grupo 7, conservando los antecedentes y archivos correspondientes a las fases anteriores. Dentro de la carpeta F3 se incorporó el notebook principal y el módulo *src/funciones_iqr.py*, utilizados para almacenar las funciones reutilizables desarrolladas en esta etapa.

El repositorio contribuye con la mantención de la trazabilidad de los cambios realizados durante el proyecto y centraliza el código, documentación y avances del grupo. De esta misma manera, la estructura utilizada facilita la continuidad del trabajo en las siguientes etapas de la asignatura.

Referencias

- Universidad Andrés Bello. (2026). *MCDI500: Apunte Fase 3*. Magíster en Ciencia de Datos e Inteligencia Artificial.
- Universidad Andrés Bello. (2026). *MCDI500 Fase 3, Semana 2: Núcleo algorítmico, eficiencia y programación orientada a objetos*. Magíster en Ciencia de Datos e Inteligencia Artificial.
- Python Software Foundation. (2026). *time — Time access and conversions*. Python Documentation. <https://docs.python.org/3/library/time.html>
- pandas development team. (2026). *pandas.Series.quantile*. pandas documentation. <https://pandas.pydata.org/docs/reference/api/pandas.Series.quantile.html>
- McKinney, W. (2010). Data structures for statistical computing in Python. *Proceedings of the 9th Python in Science Conference*, 56–61. <https://doi.org/10.25080/Majora-92bf1922-00a>